what is ELO rating in chess

The **ELO rating** in chess is a numerical system used to measure a player's skill relative to other players. It was invented by **Arpad Elo**, a Hungarian-American physics professor, and has become the standard for ranking chess players worldwide. Here's a breakdown:

---

1. How it works

- Every player has a rating (e.g., 1600, 2200, etc.).
- When two players play a game, the **expected outcome** is calculated based on their ratings.
  - If a higher-rated player wins, they gain a **small number of points**.
  - If a lower-rated player wins, they gain **more points**, and the higher-rated player loses points.
- A draw results in points being exchanged depending on the rating difference.

---

2. The formula (simplified)

The new rating is calculated as:

$$R_{\text{new}} = R_{\text{old}} + K \times (\text{Score} - \text{Expected Score})$$

- **R_old** = your current rating
- **K** = a factor that determines how fast ratings change (often 10–40)
- **Score** = 1 for a win, 0.5 for a draw, 0 for a loss
- **Expected Score** = a probability based on rating difference

For example:

- A 2000-rated player vs a 1800-rated player: the 2000 player is expected to win.
- If the 1800 player wins, they gain more points than if the 2000 player wins.

---

3. Key points

- Ratings are **relative**, not absolute.
- They change over time as players win or lose games.
- Used by **FIDE**, USCF, online platforms (like Chess.com, Lichess).
- Higher ratings indicate stronger players:
  - ~1200–1400: beginner
  - ~1800–2000: club player
  - 2200+: master
  - 2700+: elite grandmaster

Essentially, **ELO rating is a way to predict who's likely to win a chess game and track skill progression over time**.

If you want, I can also explain **why K-factors matter and how rating inflation works**—it's actually pretty interesting!

Do you want me to go into that?

how to get the expected score?

Ah! That's the core of the **ELO system**—calculating the **expected score** tells you the probability of winning a game based on the rating difference. Let's go step by step.

---

1. The formula for expected score

If Player A has rating R_A and Player B has rating R_B:

$$E_A = \frac{1}{1 + 10^{(R_B - R_A)/400}}$$

- E_A = expected score for Player A (between 0 and 1)

- R_A = rating of Player A
- R_B = rating of Player B

Similarly, Player B's expected score is:

$$E_B = \frac{1}{1 + 10^{(R_A - R_B)/400}}$$

Notice that $E_A + E_B = 1$.

---

2. Example

- Player A: 2000
- Player B: 1800

Step 1: Compute the difference

$$R_B - R_A = 1800 - 2000 = -200$$

Step 2: Divide by 400

$$-200/400 = -0.5$$

Step 3: Compute $10^{-0.5} \approx 0.316$

Step 4: Plug into formula

$$E_A = \frac{1}{1 + 0.316} = \frac{1}{1.316} \approx 0.76$$

So, **Player A is expected to score 0.76 points** (76% chance of winning).

- Player B: $E_B = 1 - 0.76 = 0.24$

---

3. How it's used

Once you know the expected score, you plug it into the rating update formula:

$$R_{\text{new}} = R_{\text{old}} + K \times (\text{Score} - E)$$

- If Player A wins: $\text{Score} = 1$ → gains points
- If Player A loses: $\text{Score} = 0$ → loses points
- If it's a draw: $\text{Score} = 0.5$

This ensures **ratings move faster if the outcome is surprising**.

If you want, I can make a **tiny table showing expected scores for different rating gaps**—it makes it much easier to see the pattern at a glance. Do you want me to do that?

how to trim first chr in R value

In **R**, if you have a character string or a vector of character strings and you want to **remove the first character**, there are several ways to do it. Here's a clear breakdown:

---

1. Using `substring()` or `substr()`

```r
x <- c("ALASKA", "PHOENIX", "LOS ANGELES")

# Remove first character
x_trimmed <- substring(x, 2)
x_trimmed
```

Output:

```
[1] "LASKA"     "HOENIX"    "OS ANGELES"
```

- `substring(x, 2)` starts from the 2nd character to the end of each string.
- `substr(x, 2, nchar(x))` works similarly:

```r
x_trimmed <- substr(x, 2, nchar(x))
```

---

2. Using `stringr::str_sub()` (tidyverse way)

```r
library(stringr)

x_trimmed <- str_sub(x, 2)
x_trimmed
```

- `str_sub(x, 2)` removes the first character from each string.
- Works nicely with vectors and in pipelines.

---

3. Using `sub()` with regular expressions

```r
x_trimmed <- sub("^.", "", x)
x_trimmed
```

- `^.` means "match the first character"
- Replace it with `""` → effectively removes it.

💡 **Tip:** If your vector is a **factor**, convert it to character first:

```r
x <- as.character(x)
x_trimmed <- substring(x, 2)
```

If you want, I can show a **one-liner in a `dplyr` pipeline** that trims the first character of a column in a data frame—very handy for cleaning airline codes like your dataset.

Do you want me to do that?

to get first chr

Ah! If you want to **get the first character** of a string (or each element in a vector) in R, here are the common ways:

---

1. Using `substring()`

```r
x <- c("ALASKA", "PHOENIX", "LOS ANGELES")

# Get first character
first_char <- substring(x, 1, 1)
first_char
```

Output:

```
[1] "A" "P" "L"
```

- `substring(x, 1, 1)` extracts from position 1 to 1 (the first character).

---

2. Using `substr()`

```r
first_char <- substr(x, 1, 1)
first_char
```

- Works the same way as `substring()` for single-character extraction.

---

3. Using `stringr::str_sub()` (tidyverse way)

```r
library(stringr)

first_char <- str_sub(x, 1, 1)
first_char
```

- `str_sub(x, 1, 1)` → first character of each string in the vector.

✅ **Tip:** You can combine this with `dplyr` to extract the first character of a column in a data frame:

```r
library(dplyr)

df <- data.frame(City = c("ALASKA", "PHOENIX", "LOS ANGELES"))

df <- df %>%
  mutate(FirstChar = substr(City, 1, 1))

df
```

Output:

City	FirstChar
ALASKA	A
PHOENIX	P
LOS ANGELES	L

If you want, I can also show a **shortcut to get both first and last character** in one go, which is handy for cleaning datasets. Do you want me to do that?

in r compute power using what symbol ?

In **R**, the symbol for **exponentiation / power** is `^`.

---

1. Basic usage

```r
# 2 raised to the power 3
2^3
```

Output:

```
[1] 8
```

---

2. Using variables

```r
x <- 5
y <- 2

x^y  # 5 squared = 25
```

---

3. Alternative: `**` operator

R also accepts `**` (like in Python):

```R
2**4
```

Output:

```
[1] 16
```

✅ Tip

- `^` is the standard R symbol.
- Can be used element-wise on vectors:

```R
c(1, 2, 3)^2
```

Output:

```
[1] 1 4 9
```

If you want, I can also show **how to combine it with `sqrt()` and other math functions** for quick calculations in R. Do you want me to do that?

Error in `[<-.data.frame` (`*tmp*` , row_num, "expected_point", value = c(NA, :
replacement has 64 rows, data has 1

Ah — this error in R is very common and happens when you try to **replace or assign a column in a data frame** with a vector that **doesn't match the number of rows** in the data frame. Let's break it down carefully.

Error explained

```r
Error in `[<-.data.frame`(`*tmp*`, row_num, "expected_point", value = c(NA, ...))
  replacement has 64 rows, data has 1
```

- Your **data frame** has only **1 row**.
- You are trying to assign a vector with **64 elements** to one column.
- R cannot recycle a length-64 vector into 1 row → it throws an error.

How to fix it

1. Check the number of rows

```r
nrow(df)        # how many rows in your data
length(new_vec) # how many elements in the vector you are assigning
```

- They must **match** unless you explicitly want to recycle (but recycling is usually unsafe here).

---

2. Assign to the entire column

If your data frame has **64 rows**, then you can do:

```r
df$expected_point <- new_vec
```

- Both must have **64 elements**.

---

3. Assign to a subset of rows

If you only want to assign some rows:

```r
row_num <- 1:64
df[row_num, "expected_point"] <- new_vec
```

- Make sure `df` has at least 64 rows, otherwise R throws the error.

---

4. If you computed `expected_point` for a single value

Sometimes this happens because you compute `expected_point` like this:

```r
expected_point <- 1 / (1 + 10^((R_B - R_A)/400))
```

- If `R_B` is a **vector** of length 64 but `R_A` is a **single value**, the result will be length 64.
- If you then try to assign it to a **data frame with 1 row**, R cannot do it.

✅ **Solution:** Either compute row-wise using `mutate()` with `dplyr`, or make sure your data frame has **64 rows**, one for each expected point.

If you want, I can rewrite your **ELO expected score calculation** in R properly so it works **row-wise for your airline dataset**, without this error.

Do you want me to do that?